jackjackbits and **claude** Keep working when the network is hostile, and make the app verifiable (…

c1ce902 · 3 months ago

121 lines (82 loc) · 12.5 KB

Preview Code Blame Raw

bitchat Privacy Assessment

Last reviewed: July 2026

Scope

- BLE discovery, mesh routing, gossip sync, private delivery, and courier behavior
- Nostr private fallback, bridge courier drops, mesh bridging, geohash channels, and notices
- CoreLocation and reverse geocoding
- Local persistence, panic wipe, logging, and App Store privacy manifests

The user-facing contract is `PRIVACY_POLICY.md`. This document records implementation-level behavior and residual risks that should be re-audited when storage or transport semantics change.

Current Posture

- The project operates no account system, analytics pipeline, advertising SDK, or project-owned messaging backend.
- Mesh transport is peer-to-peer. Optional internet features use third-party Nostr relays and can expose public content, coarse geohashes, timing, relay, and network metadata.

- Private payloads are end-to-end encrypted, but public mesh, board, bridge, and geohash content is intentionally visible to its participants.
- Local storage is bounded where practical and included in panic wipe, but it is not wholly ephemeral. The app persists the stores listed below.
- The app and share extension each bundle a privacy manifest declaring their actual required-reason API use.
- A locked device discloses less than an unlocked one, but not nothing. Notification previews are hidden by default and the app-switcher snapshot is covered; see "Locked and Seized Devices".

BLE Discovery and Metadata

Signed announces can expose:

- Nickname, persistent Noise public key, and Ed25519 signing public key
- Capability flags
- A bounded set of short direct-neighbor identifiers
- A coarse rendezvous geohash when the bridge capability is enabled

The app does not advertise the device's assigned name. iOS manages BLE address randomization; bitchat does not attempt to create a stable MAC address.

That randomization does not deliver the unlinkability it might suggest, because the application layer publishes stable identifiers above it. The 8-byte peer ID in every packet header is the first 8 bytes of the Noise static key fingerprint, so it does not rotate; announces carry the static keys themselves; and the fixed service UUID makes any bitchat device detectable as such by a passive scanner. A receiver in radio range can therefore recognise a specific device across sessions and locations, and detect that the app is in use at all. RSSI, timing, traffic volume, and radio fingerprints remain observable as well.

Ingress validates announce structure, sender binding, signatures, payload sizes, and freshness. Current-link Noise authentication is required before destructive courier handoff or strict directed delivery. Floods, queues, fragments, ingress work, and per-peer state are bounded.

Private Messaging and Courier Delivery

- Direct mesh sessions use Noise XX with X25519, ChaCha20-Poly1305, and SHA-256.
- Undelivered outgoing private messages remain in a bounded, ChaChaPoly-sealed outbox for at most 24 hours. Its key is stored in the keychain.

- Physical couriers store opaque Noise-sealed envelopes, not plaintext. Deposits have trust-tier quotas, per-depositor caps, a global cap, and at most a 24-hour lifetime.
- Spray-and-wait copies are bounded and progress cannot be replenished by replaying a deposit.
- Delivery status advances only after real transport admission or explicit relay acceptance; late failure cannot downgrade a delivered/read message.
- Panic wipe deletes the outbox, courier mail, keys, dedup state, and active transport state.

Residual risk: private-message metadata such as timing, radio adjacency, ciphertext size, rotating recipient tags, and relay connections remains observable. A compromised recipient device can disclose plaintext.

Public Gossip, Boards, and Media

- Recent signed public mesh messages are archived in Application Support for up to 6 hours so gossip sync survives a relaunch and can cross mesh partitions.
- Signed public board posts and tombstones persist until author-selected expiry, at most seven days. Stores are bounded by global and per-author quotas.
- Group metadata (name, roster, creator, epoch) persists as protected JSON; group keys live in the keychain until leave/removal/wipe.
- Voice notes and images are stored in Application Support. Incoming media has a 100 MB oldest-first quota, and all managed media — incoming and outgoing — is additionally bounded by age: a launch-time sweep deletes anything older than seven days. In-flight live captures and files reserved by a delivery or deletion in progress are exempt regardless of age. Panic wipe invalidates detached preparation work, cancels active transfers, closes live capture files, and removes the managed media tree before returning.

Public archives contain content already intended for public mesh/board distribution, but a seized unlocked device can reveal it. Group metadata and media can reveal relationships or content even when the in-memory chat timeline has gone away.

Nostr and Mesh Bridge

- BitChat's proprietary private-envelope fallback protects plaintext with secp256k1 key agreement, HKDF-SHA256, and XChaCha20-Poly1305. It is not NIP-17, NIP-44, or NIP-59 compatible and does not provide forward secrecy against later compromise of the recipient's static Nostr private key. Relays still see the recipient public-key tag, event timing and size, and network metadata.

- Bridge courier drops use a throwaway publisher key, an opaque Noise-sealed envelope, and a day-rotating recipient tag. Only a party already holding the recipient's Noise static key can compute candidate tags.
- Relay publication is considered successful only after an explicit NIP-01 `OK true` from at least one target relay. Rejected, disconnected, timed-out, or merely socket-written events stay retryable.
- When mesh bridge is enabled, public mesh messages not marked “nearby only” are signed under a per-cell Nostr identity and published to a neighborhood rendezvous geohash. Presence and public bridge traffic therefore expose a coarse area to relays and participants.
- A bridge gateway can carry signed bridge/location events and opaque courier drops for nearby mesh-only peers. It cannot validly publish a neighbor's radio-only message because the author must first sign the bridge event.
- Relays added by hand persist in local preferences (`nostr.customRelays`, at most 8, normalized on read) and are wiped on panic. An added relay names an operator someone chose to route through, so it is treated as sensitive local state rather than inert configuration. `.onion` addresses are accepted, which is the point: the four built-in relays are well-known clearnet hostnames and a filter blocking four names would otherwise end internet-delivered private messages until a new build shipped.
- Turning the Tor preference off routes relay sockets and the relay-directory fetch directly, disclosing the device IP to every relay operator including those carrying private messages. The settings UI states this while the preference is off.

Residual risk: Nostr relay retention and logging are outside project control. Public events may be copied indefinitely. Timing, coarse location, and participation can be correlated even when content is encrypted or per-cell identities are used.

Location

- When-in-use CoreLocation access computes geohash choices and bridge cells. Permission revocation stops live sampling and releases subscriptions.
- Exact coordinates are not persisted by bitchat or placed into mesh/Nostr payloads.
- Selected/bookmarked geohashes, teleport flags, and display names persist in local preferences; a fine geohash can identify a small area.
- Friendly place names use `CLGeocoder.reverseGeocodeLocation`. Apple may process the supplied location under its own privacy terms, so this operation is not accurately described as wholly on-device.

- Automatic presence is limited to lower-precision geohashes; precise posts occur through user-selected channels, notes, notices, or the bridge behavior presented in the UI.

Logging and Telemetry

- `SecureLogger` uses OSLog privacy markers and filters likely secrets. Release builds suppress debug verbosity.
- No project analytics or telemetry endpoint exists.
- Apple system logs, Nostr relays, network providers, and nearby radios can still observe operational metadata outside the project's logging layer.

Privacy Manifests

`bitchat/PrivacyInfo.xcprivacy` declares:

- UserDefaults: `CA92.1` for app-only preferences and `1C8F.1` for the shared app group
- File timestamps/metadata: `C617.1` for app-container files and `3B52.1` for user-granted files
- System boot time: `35F9.1` for elapsed-time deadlines and timers

`bitchatShareExtension/PrivacyInfo.xcprivacy` declares app-group UserDefaults reason `1C8F.1`. Both manifests declare no tracking domains and no data collection by the app developer. They must remain bundled in their respective executable bundles.

Locked and Seized Devices

The realistic compromise for many of the people this app is built for is not interception but a phone taken and, often, unlocked under coercion.

- Notification content is rendered by the system on the lock screen, so it is readable without unlocking. Message previews are therefore hidden by default: alerts state that a direct message, mention, or location-channel activity arrived, and withhold the message body, the sender's nickname, and the geohash until the app is opened. `userInfo` still carries the routing peer ID and deep link, neither of which the system displays. The preference is `notifications.hideMessagePreviews`; turning it off restores full previews.
- The window is covered on `willResignActive`, so the snapshot iOS stores for the app switcher shows a placeholder rather than an open conversation. The cover is opaque rather than blurred, and is added synchronously because the capture follows shortly after that notification. Panic wipe separately deletes snapshots already on disk.

- Clearing a mesh timeline erases the on-disk gossip archive behind it, so cleared public history is deleted rather than hidden. The echo watermark still suppresses pre-clear messages this device hears again from peers.
- Managed media is bounded by age as well as size, so a received photo does not outlive its conversation indefinitely.

Not addressed, and deliberately out of scope here:

- **No duress mechanism.** There is no decoy passphrase, no wipe-on-failed-authentication, and no biometric or passcode lock on the app itself. A coerced unlock discloses everything the device still holds. Adding one is a product decision as much as an engineering one: in some jurisdictions destroying data on demand is itself an offence, so a mode that *hides* may protect someone better than one that *destroys*, and the choice should be made deliberately rather than by default.
- **macOS gets no file-protection classes.** Every `FileProtectionType` application is inside `#if os(iOS)`; Data Protection on macOS additionally requires an entitlement. The Mac app also has no app-switcher equivalent.
- **Media is not sealed at the app layer.** It relies on the platform default protection class, which is readable once the device has been unlocked since boot. Sealing under a key with the same accessibility would not change that; only a key gated on user authentication would, and that conflicts with receiving media while locked.

Panic Wipe Coverage

The panic action clears identity/session state, preferences, location state, groups, prekeys, outbox mail, courier mail, bridge dedup state, gossip archive, board data, managed media, hand-added relays, and active subscriptions/transports. Managed media deletion completes synchronously, after active media work has been invalidated. Keychain secrets use device-only accessibility, and an install marker detects and clears app keys that survive uninstall before a later reinstall can use them. New persistent stores must add an explicit wipe hook and a regression test.

Release Review Checklist

- Reconcile every new Application Support, UserDefaults, keychain, cache, or relay write with this assessment and `PRIVACY_POLICY.md`.
- Re-scan required-reason APIs and validate both bundled `PrivacyInfo.xcprivacy` files before archive submission.
- Verify panic wipe reaches any newly added persistent store.

- Treat geohash precision, bridge-cell changes, new relay tags, and announce fields as privacy-surface changes.
- Re-run real-device Bluetooth, background/locked-device recovery, location revocation, and audio-route checks; simulators cannot validate the physical side of those behaviors.
- Check what a new notification discloses on a locked screen, and that the app-switcher snapshot is covered, whenever notification or scene-lifecycle code changes.